

Web программирование

Курс для бакалавриата

Чернышев Вячеслав



Логирование и метрики

Чернышев Вячеслав

Содержание лекции

1. FastAPI, Prometheus, Grafana
2. Логирование
3. Structlog, logstash, elasticsearch
4. Hawk

Зачем нужен мониторинг и логирование

1. Контроль состояния приложения
2. Быстро находить и исправлять неисправности
3. Анализировать поведение пользователей
4. Повышение качества и безопасности продукта

Метрики

Позволяют следить за состоянием приложения в реальном времени

- Производительность
- Задержки
- Количество запросов
- Состояние зависимостей (БД, внешние API)

Prometheus

это система мониторинга с открытым исходным кодом, которая собирает метрики и сохраняет их в time series БД



FastAPI и Prometheus

pip install prometheus_fastapi_instrumentator

```
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()

instrumentator = Instrumentator()
instrumentator.instrument(app)

@app.on_event("startup")
def _startup():
    instrumentator.expose(app)
```

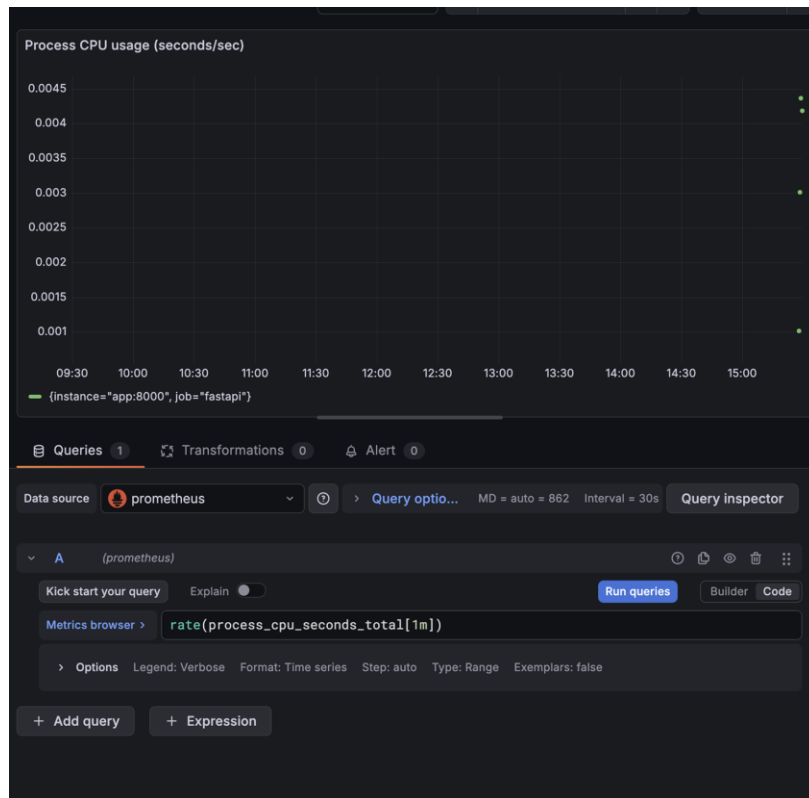
FastAPI и Prometheus

Явная запись метрики

```
RANDOM_NUMBER_TOTAL = Counter(  
    "random_number", "Total number of random number"  
)
```

```
@app.get("/compute")  
def compute():  
  
    x = sum(i * i for i in range(10000))  
    RANDOM_NUMBER_TOTAL.inc()  
    return {"result": x}
```


FastAPI и Prometheus



Prometheus query

- `node_cpu_seconds_total` - просто значение на момент времени
- `node_cpu_seconds_total[5m]` - значение за интервал времени
- `http_requests_total{method="GET"}`, `http_requests_total{method="GET"}` - фильтрация
- `cpu_usage = used / total` - арифметика
- `rate(http_requests_total[5m])` - средняя скорость изменения значения за интервал

Агрегации

- `sum()`
- `avg()`
- `min()`
- `max()`
- `count()`
- `sum by (label) (metric)`

Перцентиль

это значение метрики, ниже которого находится определённый процент измерений

- p50 - медиана
- p90 - 90 процентов измерений меньше, 10 больше

```
histogram_quantile(0.95,  
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le)  
)
```

Ключевые функции

- `increase(http_requests_total[1h])` - рост счетчика за период
- `max_over_time(node_load1[30m])` - максимум за интервал
- `avg_over_time(node_cpu_seconds_total[5m])` - среднее по диапазону

FastAPI и Prometheus

<http://localhost:8000/metrics>

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 405.0
python_gc_objects_collected_total{generation="1"} 26.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 41.0
python_gc_collections_total{generation="1"} 3.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="13",patchlevel="1",version="3.13.1"} 1.0
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{handler="/metrics",method="GET",status="2xx"} 1.0
http_requests_total{handler="none",method="GET",status="4xx"} 2.0
http_requests_total{handler="/compute",method="GET",status="2xx"} 1.0
# HELP http_requests_created Total number of requests by method, status and handler.
# TYPE http_requests_created gauge
http_requests_created{handler="/metrics",method="GET",status="2xx"} 1.76430786642884e+09
http_requests_created{handler="none",method="GET",status="4xx"} 1.764307866495574e+09
http_requests_created{handler="/compute",method="GET",status="2xx"} 1.764307908902761e+09
# HELP http_request_size_bytes Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes summary
http_request_size_bytes_count{handler="/metrics"} 1.0
http_request_size_bytes_sum{handler="/metrics"} 0.0
http_request_size_bytes_count{handler="none"} 2.0
http_request_size_bytes_sum{handler="none"} 0.0
http_request_size_bytes_count{handler="/compute"} 1.0
http_request_size_bytes_sum{handler="/compute"} 0.0
# HELP http_request_size_bytes_created Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes_created gauge
http_request_size_bytes_created{handler="/metrics"} 1.764307866428884e+09
http_request_size_bytes_created{handler="none"} 1.764307866495602e+09
http_request_size_bytes_created{handler="/compute"} 1.764307908902826e+09
# HELP http_response_size_bytes Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes summary
http_response_size_bytes_count{handler="/metrics"} 1.0
http_response_size_bytes_sum{handler="/metrics"} 3508.0
http_response_size_bytes_count{handler="none"} 2.0
http_response_size_bytes_sum{handler="none"} 44.0
http_response_size_bytes_count{handler="/compute"} 1.0
http_response_size_bytes_sum{handler="/compute"} 23.0
# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge
http_response_size_bytes_created{handler="/metrics"} 1.764307866428937e+09
http_response_size_bytes_created{handler="none"} 1.764307866495641e+09
http_response_size_bytes_created{handler="/compute"} 1.764307970098337e+09
# HELP http_request_duration_highr_seconds Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds histogram
http_request_duration_highr_seconds_bucket{le="0.01"} 4.0
http_request_duration_highr_seconds_bucket{le="0.025"} 4.0
http_request_duration_highr_seconds_bucket{le="0.05"} 4.0
http_request_duration_highr_seconds_bucket{le="0.075"} 4.0
http_request_duration_highr_seconds_bucket{le="0.1"} 4.0
http_request_duration_highr_seconds_bucket{le="0.25"} 4.0
http_request_duration_highr_seconds_bucket{le="0.5"} 4.0
http_request_duration_highr_seconds_bucket{le="0.75"} 4.0
http_request_duration_highr_seconds_bucket{le="1.0"} 4.0
http_request_duration_highr_seconds_bucket{le="1.5"} 4.0
http_request_duration_highr_seconds_bucket{le="2.0"} 4.0
```

Питоновские метрики

- `python_gc_objects_collected_total` - сколько объектов собрано GC
- `python_gc_objects_uncollectable_total` - невозможно удалить (циклические ссылки)
- `python_gc_collections_total` - сколько раз запускался GC
- `python_info{implementation="CPython",major="3",minor="13",patchlevel="1",version="3.13.1"}` 1.0 - данные о версии интерпретатора

HTTP метрики

- `http_requests_total` - количество запросов
- `http_request_duration_seconds` - задержка запросов

```
http_request_duration_seconds_bucket{handler="/metrics",le="0.1",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/metrics",le="0.5",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/metrics",le="1.0",method="GET"} 2.0
http_request_duration_seconds_bucket{handler="/metrics",le="+Inf",method="GET"} 2.0
http_request_duration_seconds_count{handler="/metrics",method="GET"} 2.0
http_request_duration_seconds_sum{handler="/metrics",method="GET"} 0.010703167004976422
```

- `http_request_duration_highr_seconds` - более точная инфа о задержке (на весь сервис)
- `http_request_size_bytes` / `http_response_size_bytes` — размер запросов/ответов

Prometheus для сбора метрик

7-lesson > 1-prometheus-grafana > ! prometheus.yml

```
1  global:
2    scrape_interval: 5s
3
4  scrape_configs:
5    - job_name: "fastapi"
6      static_configs:
7        - targets: ["app:8000"]
8
```

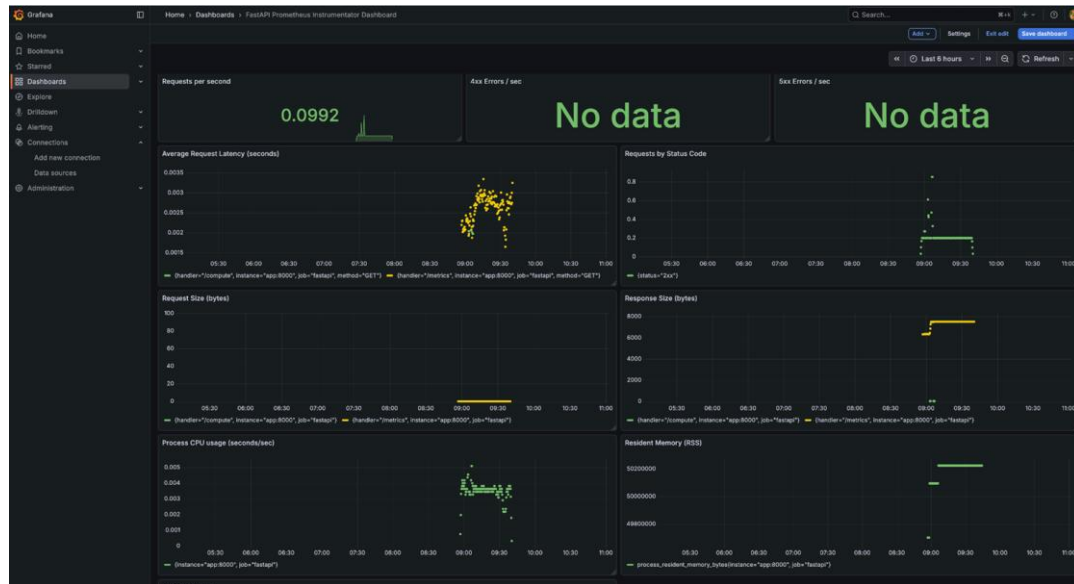
```
version: "3.9"

services:
  app:
    build: .
    container_name: fastapi_app
    ports:
      - "8000:8000"

  prometheus:
    image: prom/prometheus
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana
    container_name: grafana
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
```

Grafana



Почему print() не логирование?

- Просто выводит текст в консоль
- Не уровней важности
- Нет timestamp
- Нельзя гибко перенаправлять
- Не конфигурируем

Логирование имеет

- Уровни (debug, info, warn, error, critical)
- timestamp
- Возможность писать в файл

Логирование

- Нужны понятные логи
- Нужно понимать что делал пользователь
- Нужно отлавливать ошибки
- Отправлять логи в Elasticsearch / Loki

Базовое логирование (logging)

- Из коробки
- Питоновские библиотеки обычно его используют
- Достаточно мощный

Но есть минусы:

- Сообщения - строки
- Неудобно вставлять контекст

```
INFO:      Started server process [22917]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
2025-11-28 14:21:30,976 INFO [main] Hello endpoint called
INFO:      127.0.0.1:57455 - "GET /hello HTTP/1.1" 200 OK
```

structlog

pip install structlog

- Json формат
- сервисов несколько, логи собираются в одну систему
- легко прикручивать процессоры

Но есть минусы:

- Надо дополнительно настраивать

```
INFO: Started server process [25095]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:main:{"user_id": 1, "timestamp": "2025-11-28T11:31:14.270722Z", "level": "info", "event": "get_user_called"}
INFO:main:{"path": "/user/1", "method": "GET", "status_code": 200, "duration_ms": 2.01, "timestamp": "2025-11-28T11:31:14.271702Z", "level": "info", "event": "request_handled"}
INFO: 127.0.0.1:57855 - "GET /user/1 HTTP/1.1" 200 OK
```

structlog

```
logging.basicConfig(  
    level=logging.INFO,  
)  
  
structlog.configure(  
    processors=[  
        structlog.processors.TimeStamper(fmt="iso"),  
        structlog.processors.add_log_level,  
        structlog.processors.EventRenamer("event"),  
        structlog.processors.JSONRenderer(),  
    ],  
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),  
    context_class=dict,  
    logger_factory=structlog.stdlib.LoggerFactory(),  
)  
  
log = structlog.get_logger()
```


Middleware для логирования запросов

```
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration = (time.time() - start) * 1000

    log.info(
        "request_handled",
        path=request.url.path,
        method=request.method,
        status_code=response.status_code,
        duration_ms=round(duration, 2),
    )
    return response
```

structlog, logstash, elasticsearch

- Сервис пишет структурированные логи в файл
- Logstash читает логи из файла
- Logstash отправляет в elastic
- Дальше можно смотреть все из кибаны

structlog, logstash, elasticsearch

Настраиваем логер

```
logging.basicConfig(  
    level=logging.INFO,  
    filename="/logs/app.log",  
)  
  
structlog.configure(  
    processors=[  
        structlog.processors.TimeStamper(fmt="iso"),  
        structlog.processors.add_log_level,  
        structlog.processors.EventRenamer("event"),  
        structlog.processors.JSONRenderer(),  
    ],  
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),  
    context_class=dict,  
    logger_factory=structlog.stdlib.LoggerFactory(),  
)  
  
log = structlog.get_logger().bind(service="orders-api", env="local")
```

structlog, logstash, elasticsearch

logstash.conf

```
input {  
  file {  
    path => "/logs/app.log"  
    start_position => "beginning"  
    codec => "json"  
  }  
}  
  
filter {  
  mutate {  
    add_field => { "pipeline" => "fastapi-structlog" }  
  }  
}  
  
output {  
  stdout { codec => rubydebug }  
  
  elasticsearch {  
    hosts => ["http://elasticsearch:9200"]  
    index => "fastapi-logs-%{+YYYY.MM.dd}"  
  }  
}
```

structlog, logstash, elasticsearch

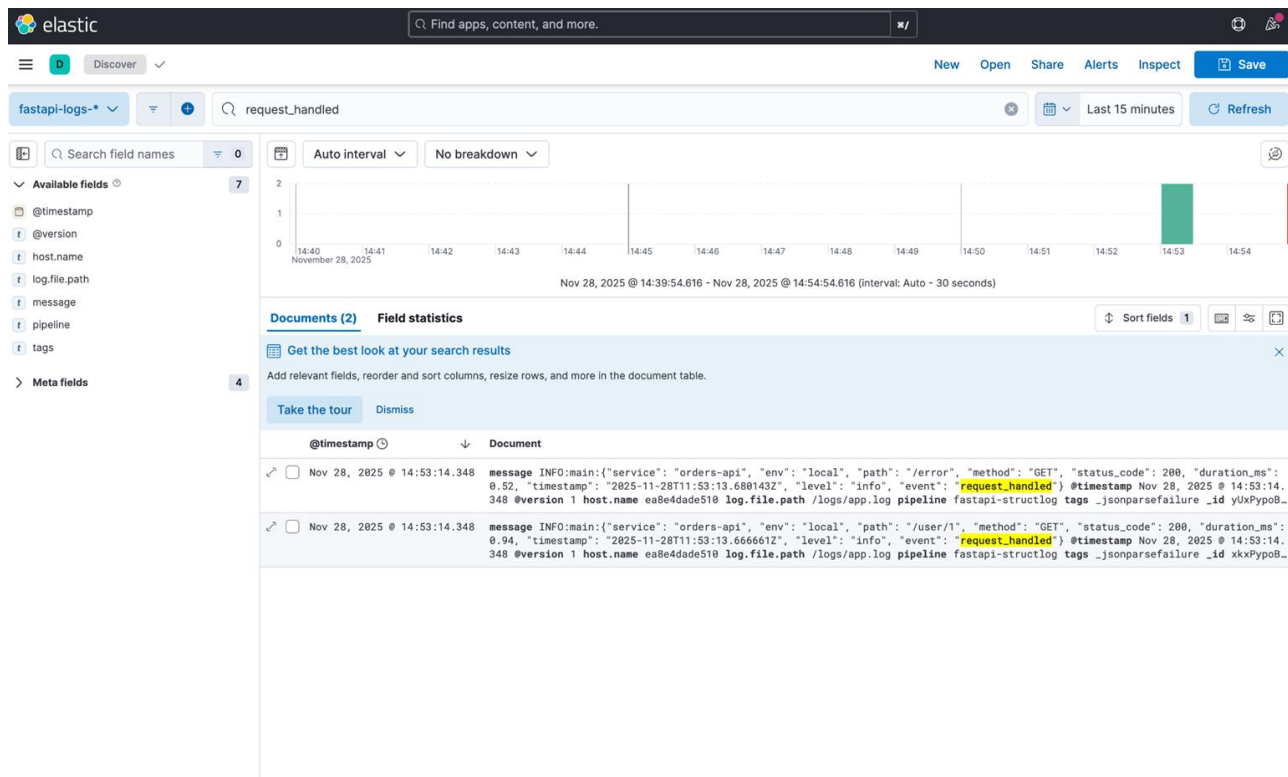
docker-compose

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.15.1
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  ports:
    - "9200:9200"
  volumes:
    - esdata:/usr/share/elasticsearch/data

logstash:
  image: docker.elastic.co/logstash/logstash:8.15.1
  container_name: logstash
  depends_on:
    - elasticsearch
  volumes:
    - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
    - logs:/logs

kibana:
  image: docker.elastic.co/kibana/kibana:8.15.1
  container_name: kibana
  depends_on:
    - elasticsearch
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  ports:
    - "5601:5601"
```

structlog, logstash, elasticsearch



StructLog vs stdlib

Когда достаточно stdlib logging

- Маленький сервис
- Мало логов
- Нет централизованного логирования
- Не надо фильтровать логи по полям

StructLog vs stdlib

Когда разумно взять StructLog

- Несколько сервисов, собирают логи в одну систему
- Нужны структурированные логи в формате json
- Добавлять контекст
- Не надо фильтровать логи по полям

Sentry

- Отлов исключений
- Мониторинг производительности
- Агрегация ошибок



Sentry

```
import sentry_sdk
from fastapi import FastAPI
from sentry_sdk.integrations.fastapi import FastAPIIntegration

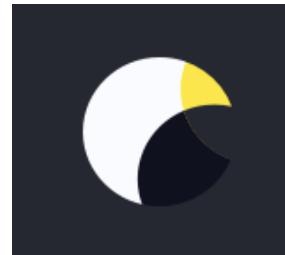
sentry_sdk.init(
    dsn="BAW_DSN",
    integrations=[FastAPIIntegration()],
    traces_sample_rate=1.0, # включает APM
)

app = FastAPI()
```



Hawk

- Трекинг ошибок
- Просмотр контекста
- Просмотр ивентов



```
pip install hawk-python-sdk[fastapi]
```

Hawk

Подключение

```
from hawk_python_sdk.modules.fastapi import HawkFastapi

from fastapi import FastAPI

app = FastAPI()

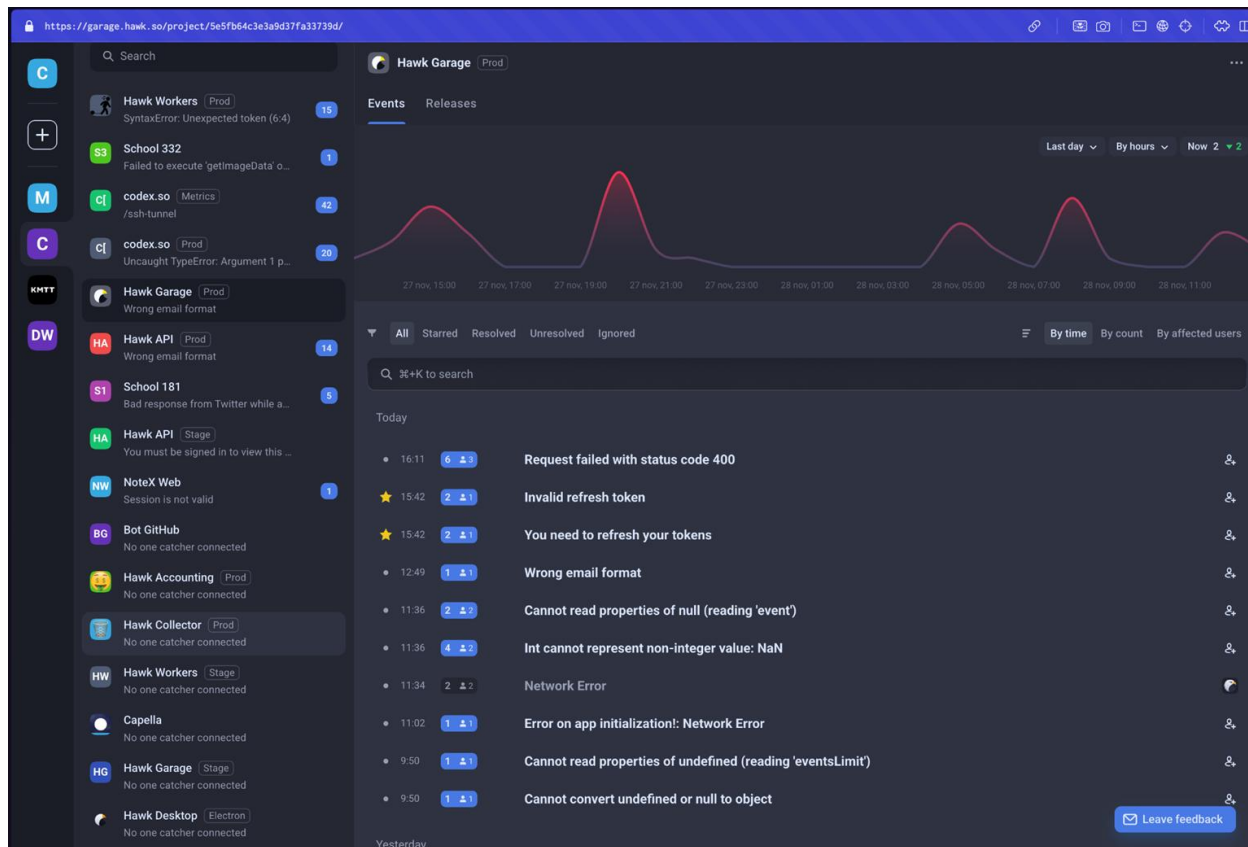
hawk = HawkFastapi({
    'app_instance': app,
    'token': 'eyJpbmRlZ3JhdGlvbklkIjoieYTAwMzJlMjUtMjFjYy00Y2'
})
```

Hawk

Явная отправка ивента

```
try:
    ...
except:
    hawk.send(ValueError("error description"), {"params": "value"}, {"id": 123})
```

Hawk



Просмотр ивента

The screenshot displays a web interface for viewing an error event. At the top, the breadcrumb navigation shows 'my-workspace / python'. The error type is 'ZeroDivisionError' with the message 'division by zero'. The file path is '/ Users / slaveeks / Documents / CodeX / hawk / catchers / tests / fastapi / main.py'. Below this are buttons for 'Resolve', 'Star', and 'Ignore'. A summary bar shows 'Overview', 'Repetitions 1', 'Days repeating', and 'Users affected 1'. The 'BACKTRACE' section lists the following frames:

- divide_ten**
/ Users / slaveeks / Documents / CodeX / hawk / catchers / tests / fastapi / main.py line 11
- run_endpoint_function**
/ Users / slaveeks / Documents / CodeX / hawk / catchers / tests / fastapi / env / lib / python3.10 / site-pa... line 212
- app**
/ Users / slaveeks / Documents / CodeX / hawk / catchers / tests / fastapi / env / lib / python3.10 / site-pa... line 301
- ... 11 more files

The 'AFFECTED USER' section shows a single user: 'User' with a profile icon and the name 'slaveeks (8)'. At the bottom, the 'FASTAPI' logo is visible, and the 'URL' field contains 'http://127.0.0.1:8000/divide-ten/0'.

Аддоны, контекст

 FASTAPI

URL

http://127.0.0.1:8000/divide-ten/0

method

GET

headers

▼

{

user-agent: ▼ "PostmanRuntime/7.50.0",

accept: ▼ "*/*",

cache-control: ▼ "no-cache",

postman-token: ▼ "f07cf9c5-dd64-4261-8d1c-d9c45e560857",

host: ▼ "127.0.0.1:8000",

accept-encoding: ▼ "gzip, deflate, br",

connection: ▼ "keep-alive"

}

COPY

cookies

▼

{}

COPY

params

▼

{}

COPY

CONTEXT

hello

world

Hawk для frontend

<https://github.com/codex-team/hawk.javascript>

```
// init Hawk Catcher instance
const hawk = new HawkCatcher({token: 'INTEGRATION_TOKEN'});

// somewhere in try-catch block or other custom place
hawk.send(new Error('Something went wrong'), {
  myOwnDebugInfo: '1234',
});
```

elastic-apm

<https://github.com/elastic/apm-agent-python?tab=readme-ov-file>

Services

Environment

All



🔍 Search transactions, errors and metrics (E.g. transaction.duration.us > 300000 AND http.response.status_cod



Last 15 minutes

Show dates

🔄 Refresh

🔍 What are these metrics?

Health ↓	Name	Environment	Transaction type	Latency (avg.)	Throughput	Error rate %
Healthy	 apm-server	production	request	 51 ms	 751.4 tpm	 0%
Healthy	 kibana	production	request	 705 ms	 75.9 tpm	 0%
Healthy	 opbeans-dotnet	production	request	 680 ms	 12.4 tpm	 2.6%
Healthy	 opbeans-ruby	production	request	 120 ms	 9.3 tpm	 1.4%
Healthy	 opbeans-java	production	request	 13 ms	 7.4 tpm	 6.0%
Healthy	 opbeans-python	production	request	 40 ms	 4.4 tpm	 1.4%
Healthy	 kibana-frontend	production	http-request	 1,233 ms	 1.0 tpm	 0%

Вопросы?